

Introduction to Theoretical Computer Science

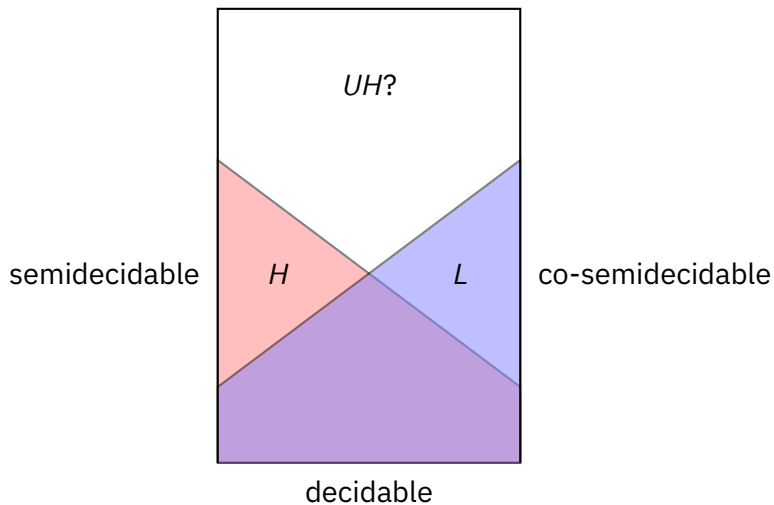
Lecture 9 [bonus, not examinable]: Arithmetical Hierarchy

Dr. Richard Mayr

University of Edinburgh

Semester 1, 2025/2026

What we have so far



Sigmas

We shall introduce notation to describe decision problems.

Sigma

The set Σ_1^0 describes all problems that can be phrased as $\{y \mid \exists x \in \mathbb{N}. R(x, y)\}$, where R is a **decidable** predicate. We can replace the $\mathbb{N}$ with any c.e. set (i.e. **type 0**).

- If a problem $P \in \Sigma_1^0$ then P is semidecidable. **Why?**
(we can enumerate all x and test $R(x, y)$, halting if true)
- If a problem P is semidecidable then $P \in \Sigma_1^0$. **Why?**

Definition: Kleene's $\mathcal{T}$ Predicate

$T(\ulcorner M \urcorner, x, y) = M$ accepts x in y steps.

If a machine M semi-decides P , then $P = \{x \mid \exists y. \mathcal{T}(\ulcorner M \urcorner, x, y)\}$

The set Π_1^0 describes all problems that can be phrased as $\{y \mid \forall x \in \mathbb{N}. R(x, y)\}$, where R is a **decidable** predicate. We can replace the $\mathbb{N}$ with any c.e. set (i.e. **type 0**).

$$\begin{aligned}\overline{\Sigma_1^0} &= \overline{\{x \mid \exists y. R(x, y)\}} \\ &= \{x \mid \neg \exists y. R(x, y)\} \\ &= \{x \mid \forall y. \neg R(x, y)\} \\ &= \Pi_1^0\end{aligned}$$

As Σ_1^0 is the set of **semidecidable** problems, Π_1^0 is the set of **co-semidecidable** problems.

Example (Empty)

Empty = $\{\ulcorner M \urcorner \mid \forall x. \forall y. \neg \mathcal{T}(\ulcorner M \urcorner, x, y)\}$ has two quantifiers.

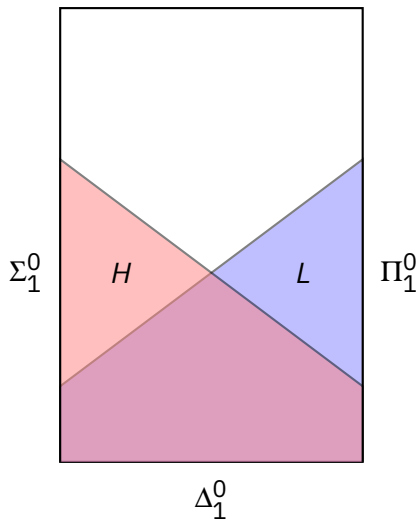
Deltas

Delta

The set Δ_1^0 describes the intersection of Σ_1^0 and Π_1^0 .

From our characterisations of Σ_1^0 and Π_1^0 , we know this describes the set of **decidable** problems.

Relabeling



Moving Higher

Definitions

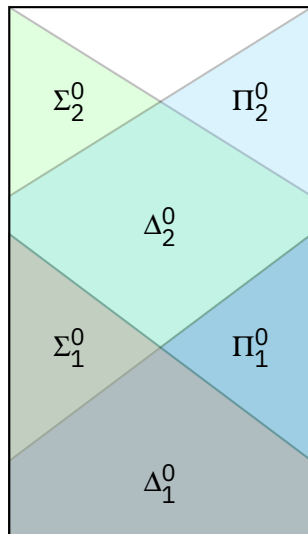
- Σ_2^0 is the set of all problems of form $\{x \mid \exists y. \forall z. R(x, y, z)\}$.
- Π_2^0 is the set of all problems of form $\{x \mid \forall y. \exists z. R(x, y, z)\}$.
- $\Delta_2^0 = \Sigma_2^0 \cap \Pi_2^0$

Note that $\Sigma_1^0, \Pi_1^0, \Delta_1^0$ are all $\subseteq \Delta_2^0$ (and therefore $\subseteq \Sigma_2^0$ and $\subseteq \Pi_2^0$). **Why?** (our R can simply “ignore” one of the parameters)

Example (Uniform Halting)

UH can be expressed as $\{\ulcorner M \urcorner \mid \forall w. \exists t. T(M, w, t)\}$.
Therefore $UH \in \Pi_2^0$.

The Arithmetical Hierarchy



An equivalent characterisation

We can define in terms of **oracles**:

- Δ_2^0 is all problems that are decidable by some TM/RM with an oracle for some (co-)semi-decidable problem.
- Σ_2^0 are all semidecidable problems by such a TM/RM.
- Π_2^0 are all co-semidecidable problems by such a TM/RM.

Building up

In general, for any $n > 1$:

- Δ_n^0 is all problems that are decidable by some TM/RM with an oracle for some problem $\in \Sigma_{n-1}^0$.
- Σ_n^0 are all semidecidable problems by such a TM/RM.
- Π_n^0 are all co-semidecidable problems by such a TM/RM.

Alternation

Equivalently Σ_n^0 are all problems that can be phrased as some **alternation** of quantifiers, starting with $\exists$:

$$\{w \mid \exists x_1. \forall x_2. \exists x_3. \forall x_4. \dots x_n. R(w, x_1, \dots, x_n)\}$$

Π_n^0 starts instead with $\forall$:

$$\{w \mid \forall x_1. \exists x_2. \forall x_3. \exists x_4. \dots x_n. R(w, x_1, \dots, x_n)\}$$

Games

Alternation of formulae are connected fundamentally with **games**.
When proving an $\exists x. \dots$, **we** have a choice of what x is. When proving a $\forall x. \dots$, **our opponent** has a choice of what x is.

Example (Pumping for CFLs)

If L is a CFL then:

$\forall p. \exists w. \forall uvxyz. \forall i. |w| \geq p \wedge |vxy| < p \wedge vy \neq \varepsilon \wedge uv^i xy^i z \in L$ Thus finding a proof via pumping that L is **not** a CFL is $\in \Sigma_3^0$.

Limitations of Oracles

Theorem

The arithmetic hierarchy is **strict**. That is, the n th level contains a language not in any level below n .

Note: H is in level 1 but not 0. Consider:

$$H_2 = \{ \langle \ulcorner M \urcorner, x \rangle \mid M, \text{ a machine with oracle for } H, \text{ halts on } x \}$$

$$H_3 = \{ \langle \ulcorner M \urcorner, x \rangle \mid M, \text{ a machine with oracle for } H_2, \text{ halts on } x \}$$

...

$$H_n = \{ \langle \ulcorner M \urcorner, x \rangle \mid M, \text{ a machine with oracle for } H_{n-1}, \text{ halts on } x \}$$

Each of these H_k -oracle machines cannot decide H_k or higher. And,
 $H_k \in \Sigma_k^0$.

Introduction to Theoretical Computer Science

Lecture 10: (Polynomial) Complexity

Richard Mayr

University of Edinburgh

Semester 1, 2025/2026

Time Complexity

We have looked into whether problems can be **computed** or not.
But are they **easy** to compute or **hard** to compute?

Time Complexity

The *time complexity* of a (deterministic) machine M that halts on all inputs is a function $f : \mathbb{N} \rightarrow \mathbb{N}$ where $f(n)$ is the maximum number of steps that M uses on any input of size n .

Example

Example

Recall that $\{0^i 1^i \mid i \in \mathbb{N}\}$ is a CFL and decidable by, e.g., a TM M_1 that given input w :

- 1 Scan w and reject if anything not in $\{\sqcup, 0, 1\}$ or 10 is found.
- 2 While there are 0s and 1s left in the tape:
 - ▶ Scan across and replace with blanks both the leftmost 0 and the rightmost 1.
- 3 If any 0s or 1s are left on the tape, reject. Else, accept.

Time complexity measure:

w	ϵ	01	$0^2 1^2$	$0^3 1^3$	$0^4 1^4$	$0^5 1^5$
$f(w)$	2	8	19	34	53	76

Big Letters

Recall from previous courses...

Big O and Ω

Let $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. Say that $f(n) \in \mathcal{O}(g(n))$ if there exists $c, n_0 > 0$ such that for all $n \geq n_0$:

$$f(n) \leq c \cdot g(n)$$

Similarly $f(n) \in \Omega(g(n))$ if:

$$f(n) \geq c \cdot g(n)$$

Example

- $f(n) = 5n^3 + 2n^2 + 22n + 6$ is $\in \mathcal{O}(n^3)$.
- M_1 's complexity is $\mathcal{O}(n^2)$.

Logarithms

Recall that comparison-based sorting has $\Omega(n \log n)$ time complexity, and we have an $\mathcal{O}(n \log n)$ algorithm.

Omitting the bases

We may safely omit the base of the logarithms here because:

$$\log_a n = \frac{\log_b n}{\log_b a}$$

Model Concerns

Addition of two numbers is $\mathcal{O}(n)$ in our RM models.

Why is this bad?

In TMs, addition is $\mathcal{O}(\log n)$ (e.g. consider binary addition).

⇒ **exponential penalty for RMs!**

If we extend our RMs with $\text{ADD}(i, j)$ $\text{SUB}(i, j)$ which instantly add/subtract R_j from/to R_i , putting the result in R_i :

Less inaccurate.. but

Now addition is $\mathcal{O}(1)$ instead of $\mathcal{O}(\log n)$, but this is a smaller inaccuracy than the exponential penalty from before.

Variations in Models

Problem?

For sorting, we counted the number of comparisons as our time measure: we assumed comparison of small numbers and big numbers take the same time.

- What about control flow or memory access costs? In RMs this can be fast, but in TMs we have to move symbol by symbol.
- As we've seen, addition has different complexities based on the model.

Question

Can we ignore these differences? How?

What counts as different?

While complexity is useful, the measures are slightly bogus:

- If a problem is $\mathcal{O}(n)$ on some model, it's surely easy on any model.

Not really: If n is a petabyte..

- If a problem is $\Omega(2^n)$ on some model, it's surely hard on any model.

Actually: There are problems that are much worse than this, but still solvable for real examples. We'll see later.

- What about something that is $\mathcal{O}(n^{10})$ or $\Omega(n^{10})$?
An $\Omega(n^{10})$ problem seems practically insoluble.

However: Maybe a new algorithm or fancy model makes it $\Omega(n^2)$?

- There's also our coefficients. If $f(n) \geq 10^{100} \log n$, that's only $\mathcal{O}(\log n)$.

However this isn't common.

Complexity Classes

Definition

Let $t : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. A *time complexity class* **TIME**($t(n)$) to be the collection of all problems that are decidable by a machine in $\mathcal{O}(t(n))$ time.

We'll give a more precise definition of time in terms of bounded machines later.

Example

Recall $A = \{0^i 1^i \mid i \in \mathbb{N}\}$. Our TM M_1 can decide this in $\mathcal{O}(n^2)$. Therefore $A \in \mathbf{TIME}(n^2)$.

Can we do better?

Can we come up with a machine M_2 that shows A is in **TIME**($t(n)$) for some $t(n)$ that is asymptotically $< n^2$?

Example

Given w input:

- ➊ Scan w left to right and reject if 10 is found.
- ➋ Repeat as long as there are 0s and 1s on the tape:
 - ➊ Scan from right to left and reject if there is an odd number of non-Xs on the tape.
 - ➋ Scan from left to right and replace every other 0 by an X, beginning from the first 0. Then, do the same for 1s.
- ➌ If neither 0s nor 1s are left, accept. Else, reject.

Steps 1, 2.1, 2.2, and 3 are all $\mathcal{O}(n)$. Step 2 runs the substeps $\mathcal{O}(\log n)$ times. So this is $\mathcal{O}(n \log n)$.

Comparing real times

Comparing the running times of M_1 and M_2 :

w	ϵ	01	$0^2 1^2$	$0^3 1^3$	$0^4 1^4$	$0^5 1^5$
$f_{M_1}(w)$	2	8	19	34	53	76
$f_{M_2}(w)$	1	15	45	63	117	141

M_2 has “better” complexity, but M_1 performs better for small n .
(M_2 will be faster for $0^{20} 1^{20}$.)

Doing better

Could we do **still better** for A ? I.e. a sub- $\mathcal{O}(n \log n)$ algorithm for A ?

Example (Two tape TMs)

The answer is no, **for a single-tape TM**. But in a two tape TM, we can copy all 0s onto the second tape and then compare the number of 0s to 1s by moving the second tape head synchronously with the first.

Polynomial Time

Definition

$$\mathbf{P} = \bigcup_{k \in \mathbb{N}} \mathbf{TIME}(n^k)$$

That is, the class of problems decidable with some (deterministic) **polynomial time** complexity.

- Problems in $\mathbf{P}$ are called *tractable*.
- The class is **robust**: “Reasonable” changes in model don’t change it, and “reasonable” translations between problems preserve membership in $\mathbf{P}$.
- Any problem not in $\mathbf{P}$ is $\Omega(n^k)$ for every k , e.g. 2^n or $2^{\sqrt{n}}$.

Outside P

Definition

A *polynomially-bounded RM* is an RM together with a polynomial (wlog n^k for some k), such that given an input w , it will always halt after executing $|w|^k$ instructions.

A problem Q is in **P** iff it is computed by polynomially-bounded RM.

Polynomial Reductions

Recall:

To prove that a problem P_2 is *hard*, show that there is an *easy* reduction from a known hard problem P_1 to P_2 .

Definition

A **polynomial reduction** from $P_1 = (D_1, Q_1)$ to $P_2 = (D_2, Q_2)$ is a **P**-computable function $f : D_1 \rightarrow D_2$ such that $d \in Q_1$ iff $f(d) \in Q_2$.

- If P_2 is in **P**, then P_1 is in **P** straightforwardly.
- Therefore: To prove that a problem P_2 is **not in P**, show that there is a **polynomial reduction** from a known **non-P** problem P_1 to P_2 .

Question: Is this more like a mapping or Turing reduction?

Apparently Intractable Problems

These problems appear to be non-**P**, so if they are, we could use them as our known non-**P** problems.

Example (Hamiltonian Path Problem)

Given a graph $G = (V, E)$, is there a path that visits every vertex in V exactly once?

We could solve this in $\mathcal{O}(|V|!)$, but this is not ideal..

Example (Timetabling)

Given students taking exams, and timetable slots for exams, is it possible to schedule the exams so that there are no clashes? It also apparently requires looking at exponentially many possible assignments.

(That's why Registry starts timetabling exams 9 weeks in advance...)

Open problem: Are they really not in **P**?

Checking

Consider *HPP* (the Hamiltonian Path Problem) or timetabling. Both are apparently **not in P**.

However..

They are **easy to check**:

Given a claimed solution, it's tractable to check if the solution is indeed a correct solution.

Theorem

Any problem that can be **checked** in polynomial time on a deterministic RM/TM can be **computed** in polynomial time on a *nondeterministic RM/TM*.

Nondeterminism

We can have nondeterministic RMs just like we have nondeterministic finite automata.

The Change

Add a special instruction $\text{MAYBE}(j)$ that will nondeterministically either do nothing or jump to I_j .

Example (generating a nondetermined number)

```
        CLEAR    $R_0$ 
beg :    MAYBE   end
        INC      0
        GOTO     beg
end :
```

Non-nondeterminism

Acceptance

An NRM **accepts** if there is *some run* (sequence of instructions through the choices) halts and accepts.

“Accepts” could mean halting, halting with 1 in R_0 or anything else.

- Nondeterminism is **NOT** probability. No randomness is involved.
- The presence of infinite runs doesn't matter if there are also accepting finite runs.
- I sometimes like to think of MAYBE as FORK: the machine forks a copy of itself which takes the jump. If any copy accepts, it signals the OS, which kills off all the others.

Question: Do NRMs have the same deciding power as RMs?

Comparing RMs and NRMs

Power

NRMs have the same deciding power as RMs, because we can use the **interleaving** technique to simulate all runs of an NRM.

Sipser has the same result for TMs.

However!

In time n , an RM can explore only $\mathcal{O}(n)$ possibilities, but an NRM can explore $2^{\mathcal{O}(n)}$ possibilities.

NRMs are **potentially exponentially faster than RMs.**

NP

Definition

Let $t : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. Define **NTIME**($t(n)$) to be the collection of all problems that are decidable by an NRM in $\mathcal{O}(t(n))$ time.

Definition

$$\mathbf{NP} = \bigcup_{k \in \mathbb{N}} \mathbf{NTIME}(n^k)$$

That is, the class of problems decidable with some nondeterministic **polynomial time** complexity.

Is *HPP* in **NP**? Nondeterministically “guess” any path and check if it is Hamiltonian ($\mathcal{O}(n)$).

A Short Aside

Can we **implement** nondeterminism or is it just a theoretical exercise?

Quantum Computing

Quantum computers can achieve a similar effect: an n -qubit computer computes on all 2^n values simultaneously.

However, one cannot really access all these 2^n values in an arbitrary way.

Not every **NP** algorithm is quantum-computable (as far as we know).
Not every problem (e.g., odd/even parity of bitstrings) can be solved faster by quantum computers.

Moreover, in practice it is hard to get many qubits..

Is NP All?

Is every exponentially-bounded problem in **NP**? probably **No!**

Tough problem

Given a machine M and input w , determine if M halts in less than $2^{|w|}$ steps.

There doesn't seem to be anything to do but run the machine M for an exponential number of steps $\Rightarrow$ *Probably* not in **NP**.

Introduction to Theoretical Computer Science

Lecture 11: NP-Completeness

Richard Mayr

University of Edinburgh

Semester 1, 2025/2026

Hardness

Definition

A problem P_1 is *polynomially reducible* to P_2 , written $P_1 \leq_P P_2$, if there is a polynomially-bounded reduction from P_1 to P_2 .

Recall:

To prove that a problem P_2 is *hard*, show that there is an *easy* reduction from a known hard problem P_1 to P_2 .

Definition

A problem P is **NP-Hard** if, for *every* $A \in \mathbf{NP}$, $A \leq_P P$

- If a problem P_1 is **NP-hard** and $P_1 \leq_P P_2$ then P_2 is **NP-Hard**.
- To prove that a problem P_2 is **NP-hard**, show that there's a *polynomial* reduction from a known **NP-hard** P_1 to P_2 .

Completeness

Question

If any **NP**-hard problem is shown to be in **P**, what does that mean?

Definition

A problem is **NP-complete** if it is both **NP**-hard and in **NP**.

Do **NP**-Complete Problems Exist?

There are **many** such problems, including *HPP* and Timetabling. In fact, almost all **NP**-problems encountered in practice are either in **P**, or **NP**-complete.

Computers and Intractability - A guide to theory of NP-completeness, M.R. Garey and D.S. Johnson, Freeman 1979 lists a whole bunch.

The original **NP**-Complete problem

The *Cook-Levin theorem* states that a particular **NP** problem, *SAT*, is **NP**-complete. The theorem is usually proved for TMs; we shall do it later for RMs.

Why Cook-Levin?

The notion of **NP**-completeness, and the theorem, were due to Stephen Cook (and partly Richard Karp)—in the West. But as with many major mathematical results of the mid-20th century, they were discovered independently in the Soviet Union, by Leonid Levin. Since the fall of the Iron Curtain made Soviet maths more accessible, we try to attribute results to both discoverers.

SAT

SAT is a very significant problem about **boolean formulae**.

The SAT Problem

Given a boolean formula ϕ over a set of boolean variables X_i , is there an **assignment** of values to X_i which **satisfies** ϕ ?

(i.e. makes ϕ true?)

Equivalently, is ϕ non-contradictory?

- $(A \vee B) \wedge (\neg B \vee C) \wedge (A \vee C)$ is satisfiable, e.g. by making A and C true. $(A \wedge B) \wedge (\neg B \wedge C) \wedge (A \wedge C)$ is not satisfiable.
- The size of a SAT problem is the number of symbols in ϕ .
- SAT is obviously in **NP**: just nondeterministically “guess” an assignment and check it.
- It's also apparently exponential in reality: no obvious way to avoid checking all possible assignments (the truth table method).

The Proof

- **The SAT problem is in NP:** Nondeterministically guess an assignment and check it in polynomial time.
- **The SAT problem is NP-Hard:** Shown by reduction from **any NP problem** to SAT.

The Reduction

Suppose $(D, Q) \in \mathbf{NP}$. We shall construct a reduction $Q \leq_P \text{SAT}$. Given an instance $d \in D$, we shall construct a formula ϕ_d which can be satisfied if its variables describe the successful executions of an NRM checking Q . This machine can be polynomially bounded, so the size of ϕ_d will be polynomial in the size of d .

The Variables

Our NRM for Q , $M = (R_0, \dots, R_{m-1}, l_0, \dots, l_{n-1})$ runs for s steps (i.e. $p(|d|)$ where d is our input and p is our polynomial bound).

Name	Meaning	How Many
C_{tj}	Program counter at step t is on l_j .	$s \cdot n$
R_{tik}	k th bit of R_i at step t .	$(2s) \cdot m \cdot s$

Why $2s$?

How big can the registers get? Running s steps of $\text{ADD}(0, 0)$ will make R_0 double s times, if it starts at $2^{|d|}$ then we need $2^{|d|+s}$ capacity. Then w.l.o.g. 2^{2s} i.e. $2s$ bits is enough.

The Formula

$$C_{00} \wedge \rho_{\text{init}} \wedge \chi_{\text{one}} \wedge \bigwedge_t \chi_t \wedge \alpha$$

Name	Meaning	How Many
χ_{one}	Program counter is in one place.	$s \cdot n^2$
χ_t	Step $t + 1$ follows from step t .	$s^2 \cdot m$
ρ_{init}	Initial register values	$m \cdot n$
α	machine accepts	s

Details

Some formulae are easy:

Program counter is in one place

$$\chi_{\text{one}} \equiv \bigwedge_t \bigvee_j \left(C_{tj} \wedge \bigwedge_{j' \neq j} \neg C_{tj'} \right)$$

Some are more tedious:

Step $t + 1$ follows from step t

$\chi_t \equiv \varphi_t \wedge \rho_t$ where φ_t models **control flow** changes and ρ_t models **register** changes.

$\varphi_t \equiv \bigvee_j (C_{tj} \wedge v_{tj})$, where v_{tj} is:

- $C_{t+1,j+1}$ if l_j is INC, ADD, or SUB.
- $C_{t+1,j+1} \vee C_{t+1,j'}$ if l_j is MAYBE(j')
- $((\bigvee_k R_{tik}) \wedge C_{t+1,j+1}) \wedge ((\bigwedge_k \neg R_{tik}) \wedge C_{t+1,j'})$
if l_j is DECJZ(i, j')

Register changes

The formulae concerning registers are very tedious, but can easily be found. See your hardware course!

Exercise

Write a formula $\rho_{tj'j}^+$, which states that at step $t + 1$, R_j will have the sum of the values in R_j and $R_{j'}$ at step t .

Despite this being **very tedious**, these formulae are polynomial ($\mathcal{O}(s^4)$) !

3SAT

- ϕ is in *conjunctive normal form* if it is of the form $\bigwedge_i \bigvee_j P_{ij}$ where each P_{ij} is a *literal* (either a variable P or negation of one $\neg P$).
- ϕ is in *k-CNF* if each clause $\bigvee_j P_{ij}$ has at most k literals.

The Problem

3SAT is the problem of whether a satisfying assignment exists for a formula in *3-CNF*.

Reduction from SAT to 3SAT is difficult, because normally converting to 3-CNF is an exponential blowup. The *Tseitin encoding* is used instead to give a not-equivalent but *equisatisfiable* formula.

Note: For TM, the Cook-Levin theorem is shown directly for 3SAT.

Clique

The *CLIQUE* problem

Given a graph $G = (V, E)$ and a number k , a *k-clique* is a k -sized subset C of V , such that every vertex in C has an edge to every other. (C forms a complete subgraph.) Decide whether G has a k -clique.

Exercise: Why is *CLIQUE* $\in$ NP?

Reducing from 3SAT, we have a formula

$$\varphi = \bigwedge_{1 \leq i \leq k} (x_{i1} \vee x_{i2} \vee x_{i3})$$

The Graph

Each x_{ij} is a vertex. Connect x_{ij} to $x_{i'j'}$ iff: $i \neq i'$ and $x_{i'j'}$ is not the negation of x_{ij} .

i.e. we connect literals in different clauses so long as they are not inconsistent.

Why does this work?

Since the vertices in one clause are disconnected, finding a k -clique amounts to finding one literal for each clause, such that they are all consistent — and so represent a satisfying assignment. Conversely, any satisfying assignment generates a k -clique.

P vs. NP

As previously mentioned, we **don't know** if **P** and **NP** are really distinct classes.

Find a polynomial time algorithm for any **NP**-hard problem and you can win yourself one million US dollars from the Clay Institute. (Also hire bodyguards because most web/banking security depends on such problems being hard.)

Many complexity theory results start with “if **P** $\neq$ **NP**...”

Example (**NP**-Intermediacy)

A problem is **NP**-Intermediate if it is in **NP** but not in **P** nor **NP**-complete.

If **P** $\neq$ **NP**, then **graph isomorphism** is such a problem

(and there aren't many others).

NP in Practice

As far as we know, **NP** problems are just hard: need exponential search, so $\mathcal{O}(p(n) \cdot 2^n)$. So how do we solve them in practice?

- Randomized algorithms are often useful. Allow algorithms to toss a coin. Surprisingly one can get randomized algorithms that solve e.g. 3SAT in time $\mathcal{O}(p(n) \cdot \frac{4}{3}^n)$.

(Why is this useful? $2^{100} \approx 10^{31}$, while $1.33^{100} \approx 10^{12}$)

Catch: (really) small probability of error!

- In many special classes (e.g. sparse graphs, or almost-complete graphs), heuristics lead to fast results. See <http://satcompetition.org/> for the state of the art.

Next time...

We'll be looking at the **boundaries** of the class **NP**, and what lays beyond. Specifically, the classes of **coNP** and **PSPACE**, as well as the *polynomial hierarchy*, analogous to the arithmetic hierarchy we've already seen, but contained entirely within **PSPACE** decidable problems.

Introduction to Theoretical Computer Science

Lecture 12: The Polynomial Hierarchy, Alternation and **PSPACE**

Richard Mayr

University of Edinburgh

Semester 1, 2025/2026

The Class **CoNP**

trick Question

Is the class **NP** closed under complement?

We don't know.

Question

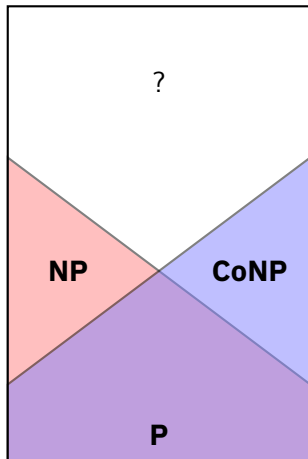
Why can't we just flip the answer, like we do for **P**?

Nondeterministic machines accept if **just one** path accepts. To flip the answer of an NRM, we'd have to accept an answer if **all paths** reject. This is no longer an (angelic) NRM (resp. NTM).

Question

What is **NP** $\cap$ **CoNP**? is it **P**? (we don't know)

What we have now



Sigmas

We shall introduce notation to describe polynomial problems.

Sigma

The set Σ_1^P describes all problems that can be phrased as $\{y \mid \exists^P x \in \mathbb{N}. R(x, y)\}$, where R is a **P-decidable** predicate and $\exists^P x \dots$ indicates that x is of size polynomial in the size of y .

- If a problem $Q \in \Sigma_1^P$ then Q is in **NP**. **Why?**
(we can “guess” an x and polynomially test $R(x, y)$)
- If a problem Q is in **NP** then $Q \in \Sigma_1^P$. **Why?**

Certificates

We can say that x is a **certificate** showing which “guesses” can be made by our NRM giving an accepting run.

So, **NP** = Σ_1^P .

Pi

The set Π_1^P describes all problems that can be phrased as $\{y \mid \forall^P x \in \mathbb{N}. R(x, y)\}$, where R is a **P-decidable** predicate and $\forall^P x \dots$ indicates that x is of size polynomial in the size of y .

$$\begin{aligned}
 \overline{\Sigma_1^P} &= \overline{\{x \mid \exists^P y. R(x, y)\}} \\
 &= \{x \mid \neg \exists^P y. R(x, y)\} \\
 &= \{x \mid \forall^P y. \neg R(x, y)\} \\
 &= \Pi_1^P
 \end{aligned}$$

As Σ_1^P is **NP**, Π_1^P is **CoNP**.

Deltas

For reasons that are unknown to me, some books have:

Delta

The set $\Delta_1^{\mathbf{P}}$ describes the intersection of $\Sigma_1^{\mathbf{P}}$ and $\Pi_1^{\mathbf{P}}$.

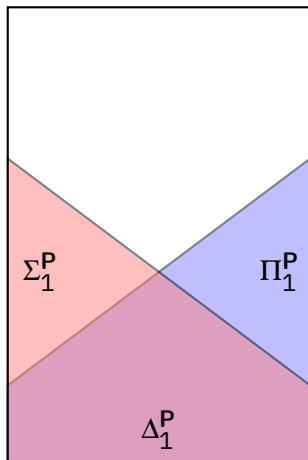
While others have

Delta

The set $\Delta_1^{\mathbf{P}}$ describes the set $\mathbf{P}$

From our characterisations of $\Sigma_1^{\mathbf{P}}$ and $\Pi_1^{\mathbf{P}}$, we have that $\Delta_1^{\mathbf{P}} \supseteq \mathbf{P}$, but we don't know if these definitions are equal.

Relabeling



Moving Higher

Definitions

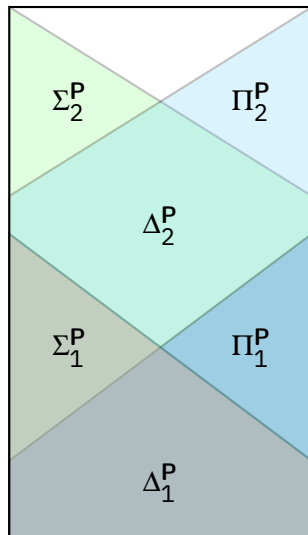
- Σ_2^P is all problems of form $\{x \mid \exists^P y. \forall^P z. R(x, y, z)\}$.
- Π_2^P is all problems of form $\{x \mid \forall^P y. \exists^P z. R(x, y, z)\}$.
- $\Delta_2^P = \Sigma_2^P \cap \Pi_2^P$

Note that $\Sigma_1^P, \Pi_1^P, \Delta_1^P$ are all $\subseteq \Delta_2^P$ (and therefore $\subseteq \Sigma_2^P$ and $\subseteq \Pi_2^P$).

Why?

(our R can simply “ignore” one of the parameters)

The Polynomial Hierarchy



An equivalent characterisation

We can define in terms of **oracles**:

- Δ_2^P is all problems that are decidable in polynomial time by some deterministic TM/RM with an $\mathcal{O}(1)$ oracle for some complete problem in Σ_1^P , i.e. it is **P** with an $\mathcal{O}(1)$ oracle for **NP**.
- Σ_2^P allows the TM/RM to be nondeterministic, i.e. it is **NP** with an $\mathcal{O}(1)$ oracle for **NP**.
- Π_2^P is **CoNP** with an oracle for **NP**.

Building up

In general, for any $n > 1$:

- Δ_n^P is all problems that are decidable by some deterministic, polynomially bounded TM/RM with an $\mathcal{O}(1)$ oracle for some problem $\in \Sigma_{n-1}^P$.
- Σ_n^P are all problems that are decidable by some **nondeterministic**, polynomially bounded TM/RM with an $\mathcal{O}(1)$ oracle for some problem $\in \Sigma_{n-1}^P$.
- Π_n^P are all problems decidable by some **co-nondeterministic**, polynomially bounded TM/RM with an $\mathcal{O}(1)$ oracle for some problem $\in \Sigma_{n-1}^P$.

Co-nondeterminism

Could also be called **demonic** nondeterminism. Like our normal (angelic) nondeterminism but only accepts if **all** paths accept.

Alternation

Alternation

Equivalently Σ_n^P are all problems that can be phrased as some **alternation** of (**P**-bounded) quantifiers, starting with $\exists^P$:

$$\{w \mid \exists^P x_1. \forall^P x_2. \exists^P x_3. \forall^P x_4. \dots x_n. R(w, x_1, \dots, x_n)\}$$

Π_n^P starts instead with $\forall^P$:

$$\{w \mid \forall^P x_1. \exists^P x_2. \forall^P x_3. \exists^P x_4. \dots x_n. R(w, x_1, \dots, x_n)\}$$

Alternating Machines

Alternating Machines combine the acceptance modes of both angelic and demonic nondeterministic machines.

Alternating Register Machines

Consider NRMs where instead of just a MAYBE instruction we have a $\text{MAYBE}^{\forall}$ instruction and a $\text{MAYBE}^{\exists}$ instruction.

- $\text{MAYBE}^{\exists}$ is a nondeterministic choice where we accept if **one** branch accepts.
- $\text{MAYBE}^{\forall}$ is a nondeterministic choice where we accept only when **both** branches accept.

Alternating Turing Machines are defined by labelling states with either $\forall$ or $\exists$.

Alternating Machines and the Polynomial Hierarchy

- The class Σ_n^P could equivalently be defined as the class of problems decided in polynomial time by an alternating machine that initially uses $\exists$ -nondeterminism, and every path in the machine swaps quantifiers (i.e. to $\forall$ or back to $\exists$) at most $n - 1$ times.
- Π_n^P is the same, except that we start with $\forall$ instead.

The class **AP**

AP is the class of all problems decidable by an alternating machine in polynomial time, without any restriction on swapping quantifiers.

AP is known to be equal to **PSPACE** (more on this in a moment)

A Fragile House of Cards

Warning

The polynomial hierarchy could collapse at any point.
(i.e. all of the classes in the PH could be equal)

We don't know that $\mathbf{P} \neq \mathbf{PSPACE}$, and the entire polynomial hierarchy is contained inside $\mathbf{AP}$ which = $\mathbf{PSPACE}$.

Wait, what's $\mathbf{PSPACE}$?

An RM/TM is *$f(n)$ -space-bounded* if it may use only $f(\text{inputsize})$ space. For TMs, space means cells on tape; for RMs, #bits in registers.

PSPACE is the class of problems solvable by polynomially-space-bounded machines.

The following inclusions follow from the definitions.

$\mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{PSPACE} \subseteq \mathbf{EXPTIME}$.

At least one of these inclusions must be strict, because we know that $\mathbf{P} \subset \mathbf{EXPTIME}$. But we don't know which one.

The common conjecture is that they are all strict.

Introduction to Theoretical Computer Science

Lecture 13: More Space Complexity

Richard Mayr

University of Edinburgh

Semester 1, 2025/2026

Logarithmic Space

Definition

$$\mathbf{L} = \mathbf{SPACE}(\log n) \quad \mathbf{NL} = \mathbf{NSPACE}(\log n)$$

where $\mathbf{SPACE}(f(n))$ (resp. $\mathbf{NSPACE}(f(n))$) are the classes of problems decidable in $f(n)$ -bounded space by a deterministic (resp. non-deterministic) Turing machine.

How can a Turing machine have a sublinear space bound?

Revised Bounded Turing Machine

Define a $f(n)$ -space-bounded Turing machine with **two tapes**:

- 1 the *input tape* is read-only, and just contains the input of size n .
- 2 the *working tape*, which is read-write and bounded by $f(n)$.

Problems in **L**

Example

$\{0^k 1^k \mid k \in \mathbb{N}\} \in \mathbf{L}$ Why?

Example

$PATH = \{\langle G, s, t \rangle \mid t \text{ reachable from } s \text{ in } \mathbf{directed} \text{ graph } G\} \in \mathbf{P}$

Is it in **L**?

- We don't know.
- **Undirected** version is in **L** (Reingold 2005), but the proof is not easy (because symmetric logspace **SL** = **L**).
- What about **NL**?

Problems in **NL**

PATH $\in$ **NL**

On input $\langle (V, E), s, t \rangle$:

- 1 store $v \leftarrow s$ on the **working tape**
- 2 repeat up to $|V| - 1$ times:
 - 3 nondeterministically 'guess' v' where $(v, v') \in E$
 - 4 if $v' = t$ accept, else set $v \leftarrow v'$
- 5 reject

Why is this in **NL**?

Question

L $\subseteq$ **NL**, but is **NL** $\subseteq$ **L**? **We don't know.**

Log-space transducers

Definition

A *log-space transducer* is a Turing machine with **three** tapes:

- 1 The **input** tape, which is read-only.
- 2 The **working** tape, which is read-write and log-bounded.
- 3 The **output** tape, which is **write-only**.

A *log-space reduction* is a reduction computable by a log-space transducer.

Hardness

Definition

A problem P_1 is *log-space reducible* to P_2 , written $P_1 \leq_L P_2$, if there is a log-space reduction from P_1 to P_2 .

Recall:

To prove that a problem P_2 is *hard*, show that there is an *easy* reduction from a known hard problem P_1 to P_2 .

Definition

A problem P is **NL-Hard** if, for *every* $A \in \mathbf{NL}$, $A \leq_L P$

- If a problem P_1 is **NL-hard** and $P_1 \leq_P P_2$ then P_2 is **NL-Hard**.
- To prove that a problem P_2 is **NL-hard**, show that there's a *log-space* reduction from a known **NL-hard** P_1 to P_2 .

Completeness

Definition

A problem is **NL-complete** if it is both **NL-hard** and in **NL**.

Example

PATH is **NL**-complete.

- We already know $PATH \in \mathbf{NL}$.
- Why is it **NL**-hard?

NL-hardness of *PATH*

Let $P \in \mathbf{NL}$. Given a nondeterministic log-space Turing machine M that computes P , we:

- 1 Construct a control-flow graph G of all the reachable configurations of M for the given input.
- 2 Ask if there is a *PATH* from the start configuration s to the accept configuration t^1 .

The transducer needs only log space on the working tape to produce the graph on the output tape.

Thus..

As $PATH \in \mathbf{NL}$ and $PATH$ is **NL**-hard, $PATH$ is **NL**-complete.

Because $PATH \in \mathbf{P}$, we conclude $\mathbf{L} \subseteq \mathbf{NL} \subseteq \mathbf{P}$!

¹W.l.o.g. we say there is just one accepting configuration.

L vs NL

We **hypothesise** that we pay an exponential time penalty when we simulate nondeterministic machines with deterministic ones, but what about **space**?

Note: We don't know that $\mathbf{NL} \not\subseteq \mathbf{L}$, so it's possible there's no penalty.

Savitch's Theorem

Define a recursive algorithm $\text{kpath}(s, t, k)$ that returns `true` iff there is a path of length k from s to t in a graph $G = (V, E)$.

- If $k = 0$, return $s = t$.
- If $k = 1$, return $(s, t) \in E$.
- If $k > 1$, for each $u \in V$:
 - ▶ If $\text{kpath}(s, u, \lfloor \frac{k}{2} \rfloor) \wedge \text{kpath}(u, t, \lceil \frac{k}{2} \rceil)$, return `true`.

kpath can compute *PATH* in $\log^2(|G|)$ space, so $\mathbf{NL} \subseteq \mathbf{L}^2$. In general $\mathbf{NSPACE}(f(n)) \subseteq \mathbf{SPACE}(f^2(n))$.

Certificates

Just as with **NP**, we can also characterise **NL** in terms of a *verifier* for *certificates* (candidate solutions):

Theorem

A problem $P \in \mathbf{NL}$ iff there is a *log-space verifier* for P -certificates.

A log-space verifier has *three tapes*:

- 1 A *input tape* that is read-only.
- 2 A *working tape* that is log-bounded.
- 3 A *certificate tape* that is *read-once* (left to right).

The size of the certificate we are verifying must be polynomial in the size of the input.

Exercise: Show that this is equivalent to our **NSPACE** definition previously.

Example

A certificate for *PATH* is a list of vertices $v_0, v_1, \dots, v_k$ forming an acyclic path from s to t in a graph $G = (V, E)$. We can check with a log-space verifier that:

- $s = v_0$
- $v_k = t$
- $(v_j, v_{j+1}) \in E$ for all $0 \leq j < k$

We only read the certificate once, left to right, and it suffices to store two nodes in our working tape, so this is log space^a.

^aNode names can be binary digits

NL vs coNL

coNL is all problems whose **complement** is in **NL**.

Immerman-Szelepcsényi Theorem

$$\mathbf{NL} = \mathbf{coNL}$$

More generally:

$$\mathbf{NSPACE}(f(x)) = \mathbf{coNSPACE}(f(x))$$

Thus:

$$\mathbf{PSPACE} = \mathbf{coPSPACE}$$

Proof of Immerman-Szelepcsényi

We prove this by showing $\overline{PATH} \in \mathbf{NL}$.

Intuition

Say I want to convince you (a verifier) that in a graph $G = (V, E)$, there is **no** path from s to t . I can do this by convincing you of the following two statements:

- 1 There are exactly $m_{|V|}$ distinct vertices reachable from s by paths of length $\leq |V|$.
- 2 The target vertex t is *not* one of those $m_{|V|}$ vertices.

So, what are the certificates?

- For Part 2, we just give a list of $m_{|V|}$ distinct vertices that are **not** t , along with a certificate for each vertex v in our list that v is reachable from s by paths of length $\leq |V|$
- For Part 1, we do **inductive counting**...

Inductive Counting

I want to convince you (the verifier) of the following:

Certify this:

There are exactly $m_{|V|}$ distinct vertices reachable from s by paths of length $\leq |V|$.

To do this, I'll make an inductive argument:

Steps

For each $k = 0, \dots, |V| - 1$, I'll show you (the verifier) that:

“if m_k vertices are reachable by paths of length $\leq k$,
then m_{k+1} vertices are reachable by paths of length $\leq k + 1$.”

Sub-certificates

Steps

For each $k = 0, \dots, |V| - 1$, I'll show you (the verifier) that:
“if m_k vertices are reachable by paths of length $\leq k$,
then m_{k+1} vertices are reachable by paths of length $\leq k + 1$.”

The certificate for each step takes the form of a sub-certificate for each vertex $v \in V$:

- If v is reachable by paths of length $\leq k + 1$, then it is just a path from s to v of length $\leq k + 1$
- If v is not reachable by paths of length $\leq k + 1$, then it is a list of m_k distinct vertices that do not have an edge to v , and a certificate for each vertex v' in our list that v' is reachable from s by paths of length $\leq k$.

There should be exactly m_{k+1} “reachable” sub-certificates (our verifier will check this).

Sub-polynomiality

We have a finer-grained notion of reduction now, so we can make distinctions smaller than **P**:

P-Completeness

A problem is **P-complete** iff it is in **P** and all problems in **P** can be log-space reduced to it.

Examples: Emptiness of CFGs, True Boolean Circuit Value, etc.

Logarithmic Hierarchy

We can imagine a *logarithmic hierarchy* like the polynomial hierarchy, i.e. the languages decided by an alternating Turing machine in logarithmic space with a bounded number of alternations.

By Immerman-Szelepcsényi, the hierarchy collapses, i.e. $\Sigma_j^L = \mathbf{NL}$ for all j . But for unbounded alternations, **AL** = **P**.

Introduction to Theoretical Computer Science

Lecture 14: λ -Calculus

Richard Mayr

University of Edinburgh

Semester 1, 2025/2026

Introduction

While Turing was thinking about **machines**, Alonzo Church was computing with a **programming language** – a precursor of Haskell – called ***λ -calculus***.

We often think of programming languages as methods to program a computer, but languages can also be thought of **as the computer** itself.

Comparing models

Church and Turing famously proved that Turing Machines and λ -calculus are equivalent in computational power.

However, λ -calculus is different from other models in that it is ***higher-order***: This means that computations (λ -terms) may take other computations as input. For TMs and RMs, we must work with encodings to achieve this.

Syntax

λ -calculus computations are expressed as *λ -terms*:

$$\begin{array}{ll} t & ::= x \quad (\text{variables}) \\ & | t_1 t_2 \quad (\text{application}) \\ & | \lambda x. t \quad (\lambda\text{-abstraction}) \end{array}$$

λ -abstraction

A λ -term $(\lambda x. y)$ can be thought of as a *function* that, given an input bound to the variable x , returns the term y .

We will give a formal definition of this in terms of *substitution* later.

For now, we will extend λ -terms with *arithmetic expressions*:

$$(\lambda x. \lambda y. (x + y) \div 2) 10 20$$

but this is *not fundamental* to the computational model. We will remove this feature later without reducing expressivity.

Higher-order functions

Function application is **left associative**:

$$f\ a\ b\ c \quad = \quad ((f\ a)\ b)\ c$$

λ -abstraction extends **as far as possible**:

$$\lambda a. f\ a\ b \quad = \quad \lambda a. (f\ a\ b)$$

All functions are unary, like Haskell. Multiple argument functions are modelled with nested λ -abstractions:

$$\lambda x. \lambda y. f\ y\ x$$

λ -calculus is **higher-order**, in that functions may be arguments to functions themselves:

$$\lambda f. \lambda g. \lambda x. f\ (g\ x)$$

α -equivalence

What is the difference between these two programs?

$$(\lambda x. \lambda x. x + x) \qquad (\lambda a. \lambda y. y + y)$$

They are **semantically** identical, but differ in the choice of **bound variable names**. Such expressions are called **α -equivalent**.

We write $e_1 \equiv_{\alpha} e_2$ if e_1 is α -equivalent to e_2 . The relation $\equiv_{\alpha}$ is an **equivalence relation**.

The process of consistently renaming variables that preserves α -equivalence is called **α -renaming** or **α -conversion**.

Substitution

A variable x is *free* in a term e if x occurs in e but is not *bound* (by a λ -abstraction) in e .

Example (Free Variables)

The variable x is free in $\lambda y. x + y$, but not in $\lambda x. \lambda y. x + y$.

A *substitution*, written $e[t/x]$, is the replacement of all *free* occurrences of x in e with the term t .

Example (Substitution on Arithmetic Expressions)

$(5 \times x + 7) [y \times 4 / x]$ is the same as $(5 \times (y \times 4) + 7)$.

Problems with substitution

Consider these two α -equivalent expressions.

$$(\lambda y. y \times x + 7) 5$$

and

$$(\lambda z. z \times x + 7) 5$$

What happens if you naïvely apply the substitution $\left[y \times 3 / x\right]$ to both expressions? You get two **non- α -equivalent** expressions!

$$(\lambda y. y \times (y \times 3) + 7) 5$$

and

$$(\lambda z. z \times (y \times 3) + 7) 5$$

This problem is called **capture**.

Variable Capture

Capture can occur for a substitution $e \left[\frac{t}{x} \right]$ whenever there is a bound variable in the term e with the same name as a free variable occurring in t .

Fortunately

It is **always possible** to avoid capture. Just **α -rename** the offending bound variable to an unused name.

β -reduction

The rule to evaluate function applications is called β -reduction:

$$(\lambda x. t) u \quad \mapsto_{\beta} \quad t [u/x]$$

β -reduction

β -reduction is a *congruence*:

$$\frac{}{(\lambda x. t) u \mapsto_{\beta} t [u/x]} \quad \frac{t \mapsto_{\beta} t'}{s t \mapsto_{\beta} s t'} \quad \frac{s \mapsto_{\beta} s'}{s t \mapsto_{\beta} s' t} \quad \frac{t \mapsto_{\beta} t'}{\lambda x. t \mapsto_{\beta} \lambda x. t'}$$

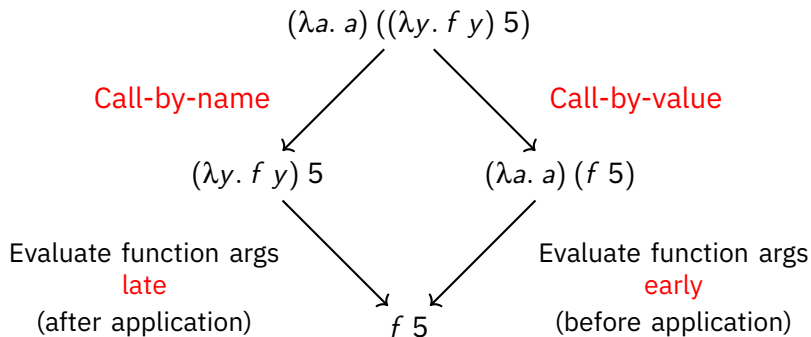
This means we can pick any reducible subexpression (called a *redex*) and perform β -reduction.

Example:

$$\begin{aligned} (\lambda x. \lambda y. f (y x)) \text{ 5 } (\lambda x. x) &\mapsto_{\beta} (\lambda y. f (y \text{ 5})) (\lambda x. x) \\ &\mapsto_{\beta} f ((\lambda x. x) \text{ 5}) \\ &\mapsto_{\beta} f \text{ 5} \end{aligned}$$

Confluence

There are often many different ways to reduce the same expression:



The Church-Rosser Theorem

If a term t β -reduces to two terms a and b , then there is a common term t' to which both a and b are β -reducible.

Equivalence

Confluence means we can define another notion of *equivalence*, which equates more than α -equivalence. Two terms are $\alpha\beta$ -*equivalent*, written $s \equiv_{\alpha\beta} t$ if they β -reduce to α -equivalent terms.

η

There is also another equation that cannot be proven from β -equivalence alone, called η -reduction:

$$(\lambda x. f\ x) \mapsto_{\eta} f$$

Adding this reduction to the system preserves confluence (and therefore uniqueness of normal forms), so we have a notion of $\alpha\beta\eta$ -*equivalence* also.

Normal Forms

A term that cannot be reduced further is called a *normal form*

Divergence

Does every term in λ -calculus have a normal form?

$$(\lambda x. x x)(\lambda x. x x)$$

Try to β -reduce this!

Uniqueness of NFs

Does any term in λ -calculus have more than one normal form?

No: consider Church-Rosser.

Making λ -Calculus Usable

In order to demonstrate that λ -calculus is actually a usable programming language, we will demonstrate how to encode booleans and natural numbers as λ -terms, along with their operations.

General Idea

We transform a data type into the type of its *eliminator*. In other words, we make a function that can serve the same purpose as the data type at its use sites.

Booleans

How do we **use** booleans? **To choose between two results!**

So, a boolean will be a function that, given two arguments, returns the first one if it is true and the second one if it is false:

$$\text{True} \equiv \lambda a. \lambda b. a$$
$$\text{False} \equiv \lambda a. \lambda b. b$$

How do we write an if statement?

$$\text{If} \equiv \lambda c. \lambda t. \lambda e. c \ t \ e$$

Example (Test it out!)

Try β -normalising `If True False True`.

Natural Numbers

How do we **use** natural numbers? **To do something n times!**

So, a natural number will be a function that takes a function f and a value x , and applies the function f to x that number of times:

$$\begin{aligned}\text{Zero} &\equiv \lambda f. \lambda x. x \\ \text{One} &\equiv \lambda f. \lambda x. f\ x \\ \text{Two} &\equiv \lambda f. \lambda x. f\ (f\ x) \\ &\dots\end{aligned}$$

How do we write Suc?

$$\text{Suc} \equiv \lambda n. \lambda f. \lambda x. f\ (n\ f\ x)$$

How do we write Add?

$$\text{Add} \equiv \lambda m. \lambda n. \lambda f. \lambda x. m\ f\ (n\ f\ x)$$

Natural Numbers

Example

Try β -normalising Suc One.

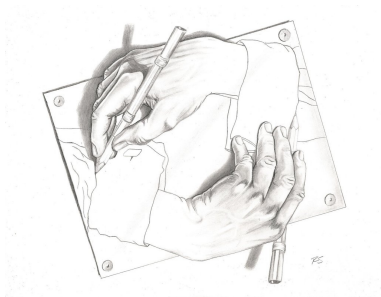
Example

Try writing a different λ -term for defining Suc.

Example

Try writing a λ -term for defining Multiply.

A Final Puzzle



Puzzle

Find a λ -term $\mathcal{Y}$ such that

$$\mathcal{Y} f \mapsto_{\beta}^* f (\mathcal{Y} f)$$

$\mathcal{Y} = \lambda f. (\lambda x. f(x x))(\lambda x. f(x x)).$

Can be used to define recursive functions, e.g. **factorial**.

Introduction to Theoretical Computer Science

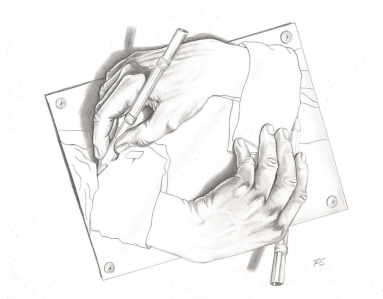
Lecture 15: Recursion and Typed λ -Calculus

Richard Mayr

University of Edinburgh

Semester 1, 2025/2026

Puzzle



Puzzle

Find a λ -term $\mathcal{Y}$ such that

$$\mathcal{Y} f \mapsto_{\beta}^* f (\mathcal{Y} f)$$

The Y Combinator

The term we are looking for is called a **fixed point combinator**. They are the way we achieve **recursion** in the λ -calculus.

Example (Recursive functions)

Exercise: Assuming a definition for $\mathcal{Y}$, as well as If, Equal, Add and Suc, define a recursive function to compute the sum of every natural number in the interval $[a, b]$.

To find $\mathcal{Y}$, we can draw inspiration from our previous diverging example:

$$(\lambda x. (x x)) (\lambda x. (x x))$$

and define $\mathcal{Y}$ as follows:

$$\mathcal{Y} \equiv (\lambda f. (\lambda x. f (x x)) (\lambda x. f (x x)))$$

Exercise: Let's demonstrate that $\mathcal{Y} g \equiv_{\beta} g (\mathcal{Y} g)$

Higher Order Logic

Originally, λ -calculus was intended for use as a term language for a logic, called *higher-order logic*. The existence of terms like $\mathcal{Y}$ poses a problem for this, as, for example:

$$\mathcal{Y} \neg \equiv_{\beta} \neg (\mathcal{Y} \neg)$$

It certainly isn't good to have a logical term that is equal to its own negation! Church solves this with *types*.

Adding Types

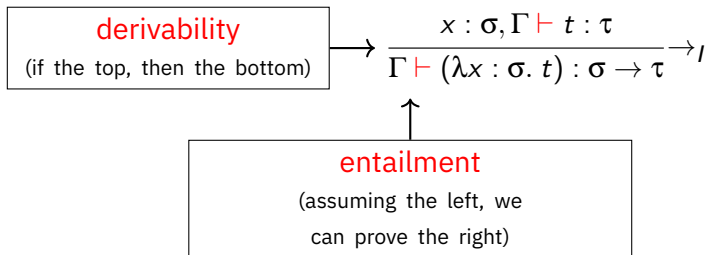
- Fix a set of *base types* (`nat`, `bool`, etc.)
- If σ and τ are types, then $\sigma \rightarrow \tau$ is a type of a *function* from σ to τ . Like Haskell, it is right-associative: $\sigma \rightarrow \tau \rightarrow \rho = \sigma \rightarrow (\tau \rightarrow \rho)$
- A λ -abstraction now additionally specifies the type of the parameter: $\lambda x : \tau. t$

Natural Deduction

Logic and Types

We can specify a logical system as a *deductive system* by providing a set of **rules** and **axioms** that describe how to prove various connectives. We can specify typing the same way!

For example, to prove a λ -abstraction $\lambda x : \sigma. t$ has type $\sigma \rightarrow \tau$, we must show that the function body t has type τ assuming x has type σ . This rule is written as:



Typing

The full set of rules for the *simply typed λ -calculus* is as follows:

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau} A \qquad \frac{x : \sigma, \Gamma \vdash t : \tau}{\Gamma \vdash (\lambda x : \sigma. t) : \sigma \rightarrow \tau} \rightarrow_I$$
$$\frac{\Gamma \vdash t : \sigma \rightarrow \tau \quad \Gamma \vdash u : \sigma}{\Gamma \vdash t u : \tau} \rightarrow_E$$

Example (Typing)

- By drawing a *proof tree*, and assuming `Add` has type `nat  $\rightarrow$  nat  $\rightarrow$  nat`, show that `( $\lambda x : \text{nat}. \text{Add } x \ x$ )` has type `nat  $\rightarrow$  nat`
- Show that our non-terminating term `( $\lambda x. x \ x$ )( $\lambda x. x \ x$ )` cannot be typed. Similarly show that `$\mathcal{Y}$` cannot be typed.

Some Results

Uniqueness of types In a given context (types for free variables), any simply typed λ -terms has at most one type. Deciding this is in **P**.

Subject reduction (type safety) Typing respects $\equiv_{\alpha\beta\eta}$, i.e. reduction does not affect a term's type.

Strong normalisation Any well-typed term evaluates in finitely many reductions to a unique irreducible term. If the type is a base type, this term is a constant.

We lost recursion!

We have seen that $\mathcal{Y}$ cannot be typed, and **strong normalisation** means that no such combinator could exist in simply typed λ -calculus.

Adding recursion back in

If we want to do general computation in our λ -calculus, we need recursion back. So, we just extend the typed λ -calculus with a new built-in feature, called **fix**:

$$\frac{\Gamma \vdash t : \tau \rightarrow \tau}{\Gamma \vdash \mathbf{fix} \ t : \tau}$$

And we extend β -reduction to unroll our recursion one step:

$$\mathbf{fix} \ (\lambda x : \tau. t) \quad \mapsto_{\beta} \quad t[\mathbf{fix} \ (\lambda x : \tau. t) / x]$$

Now we can use **fix** as we used $\mathcal{Y}$ in our untyped setting.

Total Programming

Some type-theoretic languages (Agda, Idris) **avoid** adding general recursion to their underlying λ -calculus. Let's talk about why they did that!

Product Types

Lets extend our simple lambda calculus with some other **composite types**, such as *product types* or tuples:

$$\tau_1 \times \tau_2$$

We won't have type declarations, named fields or anything like that. More than two values can be combined by nesting products, for example a three dimensional vector:

$$\text{nat} \times (\text{nat} \times \text{nat})$$

Constructors and Eliminators

We can **construct** a product type similarly to Haskell tuples:

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1, e_2) : \tau_1 \times \tau_2} \times_I$$

The only way to extract each component of the product is to use the `fst` and `snd` eliminators:

$$\frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \text{fst } e : \tau_1} \times_{E1} \qquad \frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \text{snd } e : \tau_2} \times_{E2}$$

Semantics

We extend our notion of β -reduction to describe how these new built-in features evaluate:

$$\text{fst } (v_1, v_2) \mapsto_{\beta} v_1 \qquad \text{snd } (v_1, v_2) \mapsto_{\beta} v_2$$

Unit Types

Currently, we have no way to express a type with just **one** value. This may seem useless at first, but it becomes useful in combination with other types.

We'll introduce a new base type, **1**, pronounced *unit*, that has exactly one inhabitant, written `()`:

$$\frac{}{\Gamma \vdash () : \mathbf{1}}$$

Disjunctive Composition

We can't, with just our product types, express a type with exactly **three** values.

Example (Trivalued type)

```
data TrafficLight = Red | Amber | Green
```

In general we want to express data that can be **one** of multiple **alternatives**, that contain different bits of data.

Example (More elaborate alternatives)

```
type Length = Int
type Angle = Int
data Shape = Rect Length Length
           | Circle Length | Point
           | Triangle Angle Length Length
```

Sum Types

We will use *sum types* to express the possibility that data may be one of two forms.

$$\tau_1 + \tau_2$$

This is similar to the Haskell `Either` type.

Our `TrafficLight` type can be expressed (grotesquely) as a sum of units:

$$\text{TrafficLight} \simeq \mathbf{1} + (\mathbf{1} + \mathbf{1})$$

Constructors and Eliminators for Sums

To make a value of type $\tau_1 + \tau_2$, we invoke one of two **constructors**:

$$\frac{\Gamma \vdash e : \tau_1}{\Gamma \vdash \text{InL } e : \tau_1 + \tau_2} +I1 \qquad \frac{\Gamma \vdash e : \tau_2}{\Gamma \vdash \text{InR } e : \tau_1 + \tau_2} +I2$$

We can branch based on which alternative is used using **pattern matching**:

$$\frac{\Gamma \vdash e : \tau_1 + \tau_2 \quad x : \tau_1, \Gamma \vdash e_1 : \tau \quad y : \tau_2, \Gamma \vdash e_2 : \tau}{\Gamma \vdash (\mathbf{case } e \mathbf{ of InL } x \rightarrow e_1; \text{InR } y \rightarrow e_2) : \tau} +E$$

Examples

Example (Traffic Lights)

Our traffic light type has three values as required:

$$\text{TrafficLight} \simeq \mathbf{1} + (\mathbf{1} + \mathbf{1})$$

$$\text{Red} \simeq \text{InL } ()$$

$$\text{Amber} \simeq \text{InR } (\text{InL } ())$$

$$\text{Green} \simeq \text{InR } (\text{InR } ())$$

Semantics

(case (lnL v) of lnL $x \rightarrow e_1$; lnR $y \rightarrow e_2$) $\mapsto_{\beta}$ $e_1[v/x]$

(case (lnR v) of lnL $x \rightarrow e_1$; lnR $y \rightarrow e_2$) $\mapsto_{\beta}$ $e_2[v/y]$

The Empty Type

We add another type, called **0**, that has **no** inhabitants. Because it is empty, there is no way to construct it.

We do have a way to eliminate it, however:

$$\frac{\Gamma \vdash e : \mathbf{0}}{\Gamma \vdash \text{absurd } e : \tau} \mathbf{0}_E$$

If I have a variable of the **empty** type in scope, we must be looking at an expression that will **never** be evaluated. Therefore, we can assign any type we like to this expression, because it will never be executed.

Examining our Types

Lets look at the rules for typed lambda calculus extended with sums and products:

$$\begin{array}{c} \frac{\Gamma \vdash e : 0}{\Gamma \vdash \text{absurd } e : \tau} \quad \frac{}{\Gamma \vdash () : 1} \\[10pt] \frac{\Gamma \vdash e : \tau_1}{\Gamma \vdash \text{lnL } e : \tau_1 + \tau_2} \quad \frac{\Gamma \vdash e : \tau_2}{\Gamma \vdash \text{lnR } e : \tau_1 + \tau_2} \\[10pt] \frac{\Gamma \vdash e : \tau_1 + \tau_2 \quad x : \tau_1, \Gamma \vdash e_1 : \tau \quad y : \tau_2, \Gamma \vdash e_2 : \tau}{\Gamma \vdash (\textbf{case } e \textbf{ of lnL } x \rightarrow e_1; \text{lnR } y \rightarrow e_2) : \tau} \\[10pt] \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1, e_2) : \tau_1 \times \tau_2} \quad \frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \text{fst } e : \tau_1} \quad \frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \text{snd } e : \tau_2} \\[10pt] \frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1 e_2 : \tau_2} \quad \frac{x : \tau_1, \Gamma \vdash e : \tau_2}{\Gamma \vdash \lambda x. e : \tau_1 \rightarrow \tau_2} \end{array}$$

Squinting a Little

Lets remove all the **terms**, leaving just the types and the contexts:

$$\begin{array}{c} \frac{\Gamma \vdash 0}{\Gamma \vdash \tau} \qquad \frac{}{\Gamma \vdash 1} \\[10pt] \frac{\Gamma \vdash \tau_1}{\Gamma \vdash \tau_1 + \tau_2} \qquad \frac{\Gamma \vdash \tau_2}{\Gamma \vdash \tau_1 + \tau_2} \\[10pt] \frac{\Gamma \vdash \tau_1 + \tau_2 \quad \tau_1, \Gamma \vdash \tau \quad \tau_2, \Gamma \vdash \tau}{\Gamma \vdash \tau} \\[10pt] \frac{\Gamma \vdash \tau_1 \quad \Gamma \vdash \tau_2}{\Gamma \vdash \tau_1 \times \tau_2} \qquad \frac{\Gamma \vdash \tau_1 \times \tau_2}{\Gamma \vdash \tau_1} \qquad \frac{\Gamma \vdash \tau_1 \times \tau_2}{\Gamma \vdash \tau_2} \\[10pt] \frac{\Gamma \vdash \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash \tau_1}{\Gamma \vdash \tau_2} \qquad \frac{\tau_1, \Gamma \vdash \tau_2}{\Gamma \vdash \tau_1 \rightarrow \tau_2} \end{array}$$

Does this resemble anything you've seen before?

A surprising coincidence!

Types are exactly the same structure as *intuitionistic logic*:

$$\begin{array}{c} \frac{\Gamma \vdash \perp}{\Gamma \vdash P} \qquad \overline{\Gamma \vdash \top} \\[10pt] \frac{\Gamma \vdash P_1}{\Gamma \vdash P_1 \vee P_2} \qquad \frac{\Gamma \vdash P_2}{\Gamma \vdash P_1 \vee P_2} \\[10pt] \frac{\Gamma \vdash P_1 \vee P_2 \quad P_1, \Gamma \vdash P \quad P_2, \Gamma \vdash P}{\Gamma \vdash P} \\[10pt] \frac{\Gamma \vdash P_1 \quad \Gamma \vdash P_2}{\Gamma \vdash P_1 \wedge P_2} \qquad \frac{\Gamma \vdash P_1 \wedge P_2}{\Gamma \vdash P_1} \qquad \frac{\Gamma \vdash P_1 \wedge P_2}{\Gamma \vdash P_2} \\[10pt] \frac{\Gamma \vdash P_1 \rightarrow P_2 \quad \Gamma \vdash P_1}{\Gamma \vdash P_2} \qquad \frac{P_1, \Gamma \vdash P_2}{\Gamma \vdash P_1 \rightarrow P_2} \end{array}$$

This means, if we can construct a **program** of a certain **type**, we have also created a constructive **proof** of a certain **proposition**.

The Curry-Howard Correspondence

This correspondence goes by many names, but is usually attributed to **Haskell Curry** and **William Howard**.

It is a *very deep* result:

Programming	Logic
Types	Propositions
Programs	Proofs
Evaluation	Proof Simplification

It turns out, no matter what logic you want to define, there is always a corresponding λ -calculus, and vice versa.

Constructive Logic	Typed λ -Calculus
Classical Logic	Continuations
Modal Logic	Monads
Linear Logic	Linear Types, Session Types
Separation Logic	Region Types

Examples

Example (Commutativity of Conjunction)

$$\begin{aligned} \text{andComm} &: A \times B \rightarrow B \times A \\ \text{andComm} &= \lambda p. (\text{snd } p, \text{fst } p) \end{aligned}$$

This proves $A \wedge B \rightarrow B \wedge A$.

Example (Transitivity of Implication)

$$\begin{aligned} \text{transitive} &: (A \rightarrow B) \rightarrow (B \rightarrow C) \rightarrow (A \rightarrow C) \\ \text{transitive} &= \lambda f \lambda g \lambda x. g (f x) \end{aligned}$$

Transitivity of implication is just **function composition**.

Caveats

All functions we define have to be **total and terminating**.

Otherwise we get an **inconsistent** logic that lets us prove false things:

$$\begin{aligned} proof_1 &: P = NP \\ proof_1 &= proof_1 \end{aligned}$$

$$\begin{aligned} proof_2 &: P \neq NP \\ proof_2 &= proof_2 \end{aligned}$$

This is why Agda and Idris avoid adding **fix**.

Most common calculi correspond to **constructive** logic, not **classical** ones, so principles like the **law of excluded middle** or **double negation elimination** do **not** hold:

$$\neg\neg P \rightarrow P$$